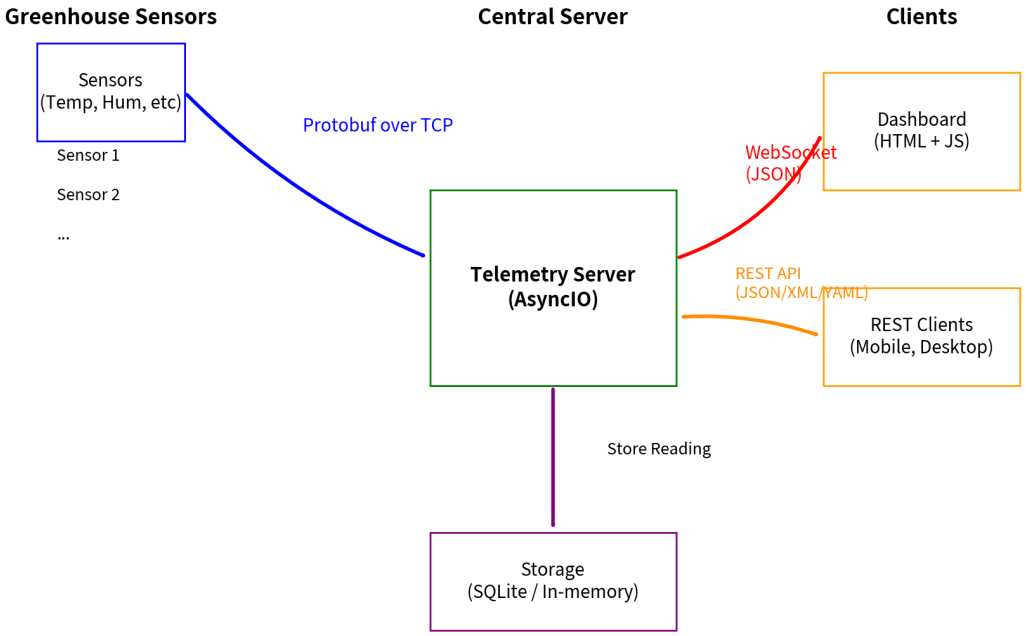
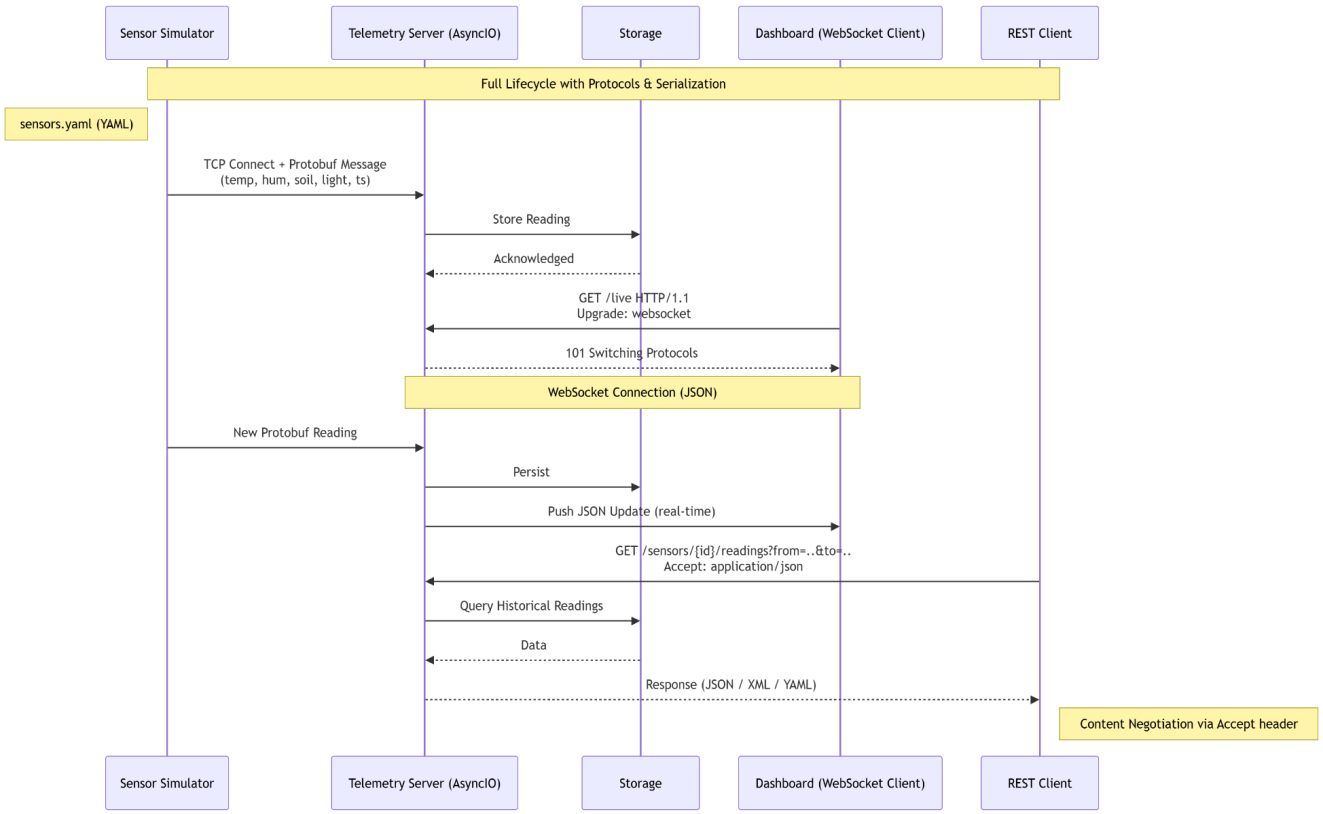
System Architecture Diagram - IoT Telemetry Pipeline





Entity Schemas:

Sensor:

Field	Type	Description	Constraints
sensor_id	String	Unique identifier	Primary Key, UUID or slug
name	String	Human-readable name	e.g., "Greenhouse A - Soil 1"
location	String	Greenhouse/farm location	Optional
type	String	Sensor type	temperature, humidity, etc.
reporting_interval	Integer	Seconds between reports	Default: 15
registered_at	DateTime	When sensor was registered	Auto-generated

Reading:

Field	Type	Description
reading_id	Integer	Auto-increment ID
sensor_id	String	Reference to Sensor
timestamp	DateTime	Time of reading
temperature	Float	°C
humidity	Float	Percentage
soil_moisture	Float	Percentage
light_level	Float	percentage

Client session:

Field	Type	Description
session_id	String	Cookie value

client_info	**String**	**Optional (User-Agent, IP, etc.)**
created_at	**DateTime**	**Session start time**

.proto file

```
package telemetry;

message SensorReading {
    string sensor_id = 1;
    int64 timestamp = 2;

    float temperature = 3;
    float humidity = 4;
    float soil_moisture = 5;
    float light_level = 6;
}

message RegisterSensor {
    string sensor_id = 1;
    string name = 2;
    string location = 3;
    string type = 4;
    int32 reporting_interval = 5;
}
```

The internal communication between sensors and the server uses the SensorReading message defined in Protobuf for efficiency. The Reading and Sensor entities are stored in SQLite and exposed through the REST API with content negotiation.

D.

Item	**Description**
Method	GET
Path	`/sensors`
Request Body	None
Query Params	None

Response Body	List of Sensor objects (JSON)
Status Codes	`200 OK`, `500 Internal Server Error`
Example	**Request:** `GET /sensors` **Response (JSON):** `200 OK`

Item	**Description**
Method	POST
Path	`/sensors`
Request Body	Sensor registration data
Response Body	Created Sensor object
Status Codes	`201 Created`, `400 Bad Request`, `409 Conflict`
Example	**Request:** `POST /sensors RequestBody{"sensor_id": "gh-b-03", "name": "Soil Sensor 3", "location": "Greenhouse B", "sensor_type": "soil_moisture","reporting_interval": 60}` **Response:** `201 Created` + created sensor object

Item	**Description**
Method	GET
Path	`/sensors/{sensor_id}/readings`
Request Body	None
Query Params	`from` (ISO datetime), `to` (ISO datetime) – optional
Response Body	Array of Reading objects

Status Codes	`200 OK`, `404 Not Found`, `400 Bad Request`
Example	**Request:** `GET /sensors/gh-a-01/readings?from=2026-05-08T10:00:00&to=2026-05-09T18:00:00` **Response:** `200 OK` `json[ { "timestamp": "2026-05-09T14:30:00", "temperature": 29.4, "humidity": 62.1, "soil_moisture": 45.8, "light_level": 13400 }]`

Item	Description
Method	GET
Path	`/sensors/{sensor_id}/readings`
Request Body	None
Query Params	`from` (ISO datetime), `to` (ISO datetime) – optional
Response Body	Array of Reading objects
Status Codes	`200 OK`, `404 Not Found`, `400 Bad Request`
Example	**Request:** `GET /sensors/gh-a-01/readings?from=2026-05-08T10:00:00&to=2026-05-09T18:00:00` **Response:** `200 OK` `json[ { "timestamp": "2026-05-09T14:30:00", "temperature": 29.4,"humidity": 62.1, "soil_moisture": 45.8, "light_level": 13400 }]`

Item	Description
Method	DELETE
Path	`/sensors/{sensor_id}`
Request Body	None

Response Body	None (or success message)
Status Codes	`204 No Content`, `404 Not Found`
Example	**Request:** `DELETE /sensors/gh-a-01` **Response:** `204 No Content`

e.

Sensor Simulator Configuration

```
sensors:
  - sensor_id: "gh-a-temp-01"
    name: "Greenhouse A - Temperature"
    location: "Greenhouse A"
    sensor_type: "temperature"
    reporting_interval: 15    # seconds

  - sensor_id: "gh-a-hum-02"
    name: "Greenhouse A - Humidity"
    location: "Greenhouse A"
    sensor_type: "humidity"
    reporting_interval: 20

  - sensor_id: "gh-b-soil-03"
    name: "Greenhouse B - Soil Moisture"
    location: "Greenhouse B"
    sensor_type: "soil_moisture"
    reporting_interval: 30

  - sensor_id: "gh-b-light-04"
    name: "Greenhouse B - Light Level"
    location: "Greenhouse B"
    sensor_type: "light"
    reporting_interval: 25
```

f . The entire system is built using **AsyncIO** to handle concurrency without threads.

Main Coroutines :

1. **Sensor Simulator Tasks**
    - One asyncio.Task per sensor.
    - Each task reads from sensors.yaml, sleeps for its reporting_interval, generates realistic readings, and sends Protobuf messages over TCP.
2. **Telemetry Server**

    - asyncio.start_server() to accept multiple sensor TCP connections concurrently.
    - One handle_sensor_client() coroutine per connected sensor.
3. **REST API Server**
    - All request handlers are async.
4. **WebSocket Handler**
    - One persistent connection handler per connected dashboard/client.

Communication:

- Sensors send data → Server stores in SQLite.
- New readings are broadcast to all connected WebSocket clients via asyncio.Queue.
- A background task periodically cleans old sessions.

Slow Consumer Handling:

If a WebSocket client is slow, the server uses a bounded queue. If the queue fills up, older readings are dropped for that client to prevent memory exhaustion.

g. **Failure Handling**

The system is designed to be resilient to common failure scenarios. When a sensor disconnects unexpectedly, the server detects the closed TCP connection, logs the event, and continues operating normally with other sensors, the sensor simulator will automatically attempt to reconnect on its next reporting cycle. Malformed or invalid Protobuf messages are caught using try-except blocks around ParseFromString(), with the server logging a warning and closing the faulty connection gracefully without crashing. In case of storage failures (e.g., SQLite errors), database operations are wrapped in exception handlers; the server returns appropriate HTTP status codes such as 503 Service Unavailable for REST requests while attempting to keep the system responsive. For slow or unresponsive WebSocket clients, a bounded queue is used per connection, old messages are dropped if the client cannot keep up, preventing memory exhaustion . Invalid REST requests (missing sensor, bad date range, etc.) return clear 400 Bad Request or 404 Not Found responses. All errors are properly logged using Python's logging module to aid debugging and monitoring.